Assignment-1

Student Name: Rashid Khan
Branch: B.E./C.S.E.
Semester: 6th
Subject Name: Full Stack II
Subject Code: 23CSH-309

UID: 23BCS12551
Section/Group: KRG-3B
Date of Performance: 1/02/2026

1. **Aim:** The aim of this project is to design and develop a Daily Water Tracker feature as part of the EcoTrack wellness dashboard using React.js. The application enables users to log and monitor their daily water intake, set a personal hydration goal, persist data across sessions, and receive a motivational health tip — all within a protected, multi-page React application.

2. **Objective:**

- Implement client-side routing using React Router with protected routes that redirect unauthenticated users to the login page.
- Build stateful UI components using React's useState and useEffect hooks to manage water count and daily goal.
- Persist user data (water count) across browser sessions using localStorage.
- Integrate a third-party REST API (adviceslip.com) to fetch and display a daily health tip with proper loading and error states.
- Prevent unnecessary re-renders of the CounterDisplay component using React.memo and useCallback for performance optimization.
- Create a modular, component-based project structure separating pages, components, and utility logic.

3. **Tools & Technologies Used:**

Tool	Purpose
Create React App / Vite	Project scaffolding and development server
Node.js & npm	Package manager and JavaScript runtime
VS Code	Code editor
Browser DevTools	Debugging, console logging, and network inspection

Tool / Technology	Purpose / Usage
React.js 18	Core UI library for building component-based interfaces
React Router DOM v6	Client-side routing — /login, /dashboard, /dashboard/water
React Hooks	useState, useEffect, useCallback for state and side effects
React.memo	Performance optimization — prevents unnecessary re-renders
localStorage (Web API)	Persisting water count and auth token across sessions
Fetch API	Calling the external health tip REST API
adviceslip.com API	Third-party API that returns random health/advice tips
corsproxy.io	CORS proxy to allow browser-safe API calls
JavaScript (ES6+)	Core language — arrow functions, destructuring, modules
CSS-in-JS (inline styles)	Component-level styling without external CSS files

4. Flow of Data:

4.1 Authentication Flow

- User visits the app → React Router checks if eco_token exists in localStorage.
- If token is absent, ProtectedRoute redirects the user to /login.
- User clicks 'Login' → login() utility stores a fake token in localStorage.
- User is then navigated to /dashboard.
- Logout clears the token from localStorage and redirects to /login.

4.2 Water Tracker State Flow

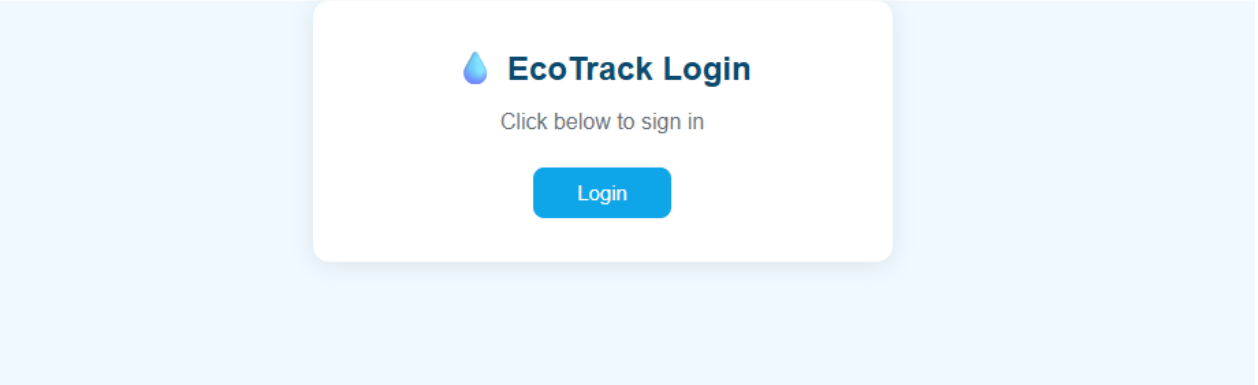
- On mount, WaterTrackerPage reads eco_water_count from localStorage (default: 0).
- count state is initialized from localStorage using the lazy initializer in useState.
- User clicks + → addWater (useCallback) → setCount increments count.
- User clicks – → removeWater (useCallback) → setCount decrements count (min: 0).
- User clicks Reset → reset (useCallback) → setCount resets to 0.
- useEffect watches count → writes updated value to localStorage on every change.
- CounterDisplay receives count and goal as props and renders the progress display.
- Since CounterDisplay is wrapped in React.memo, it only re-renders when count or goal actually changes.

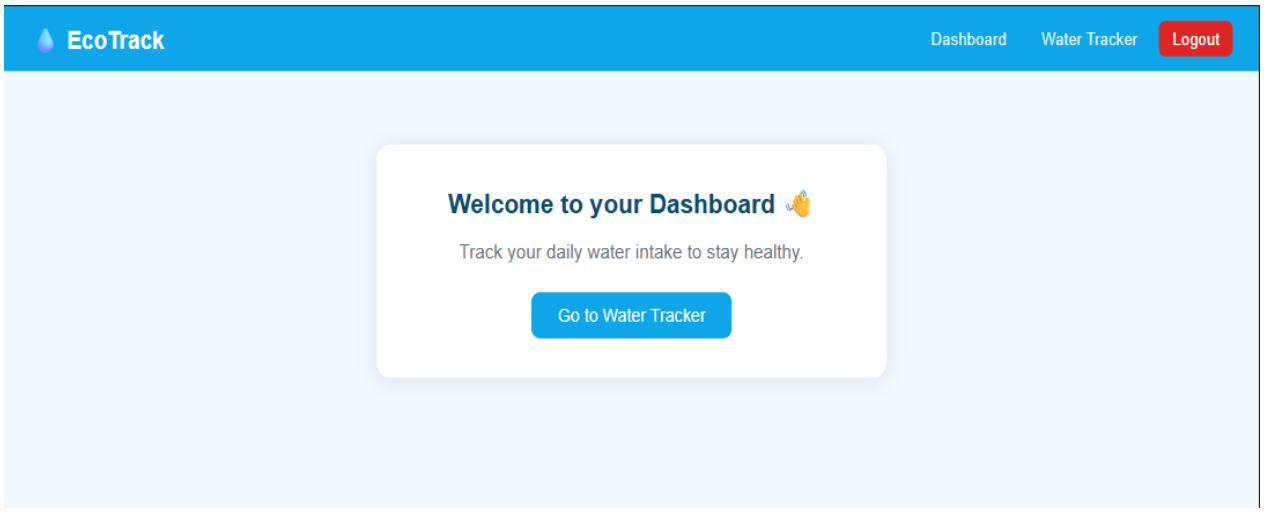
4.3 API Data Flow

- On component mount, HealthTip component sets loading = true and calls the adviceslip API via corsproxy.io.
- Fetch API sends a GET request → response is parsed as JSON → tip text is extracted from data.slip.advice.
- On success: tip state is updated and loading is set to false.
- On failure: error state is set and loading is set to false. An error message is displayed to the user.

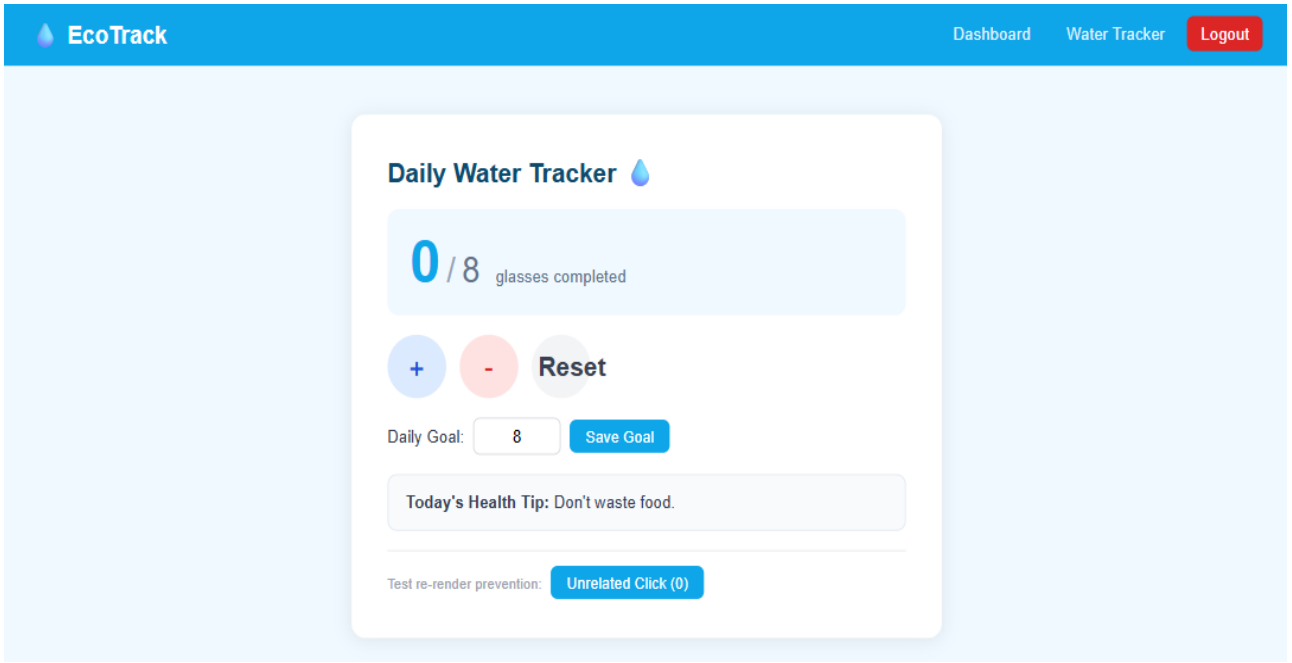
5. Output:

5.1. Login Page

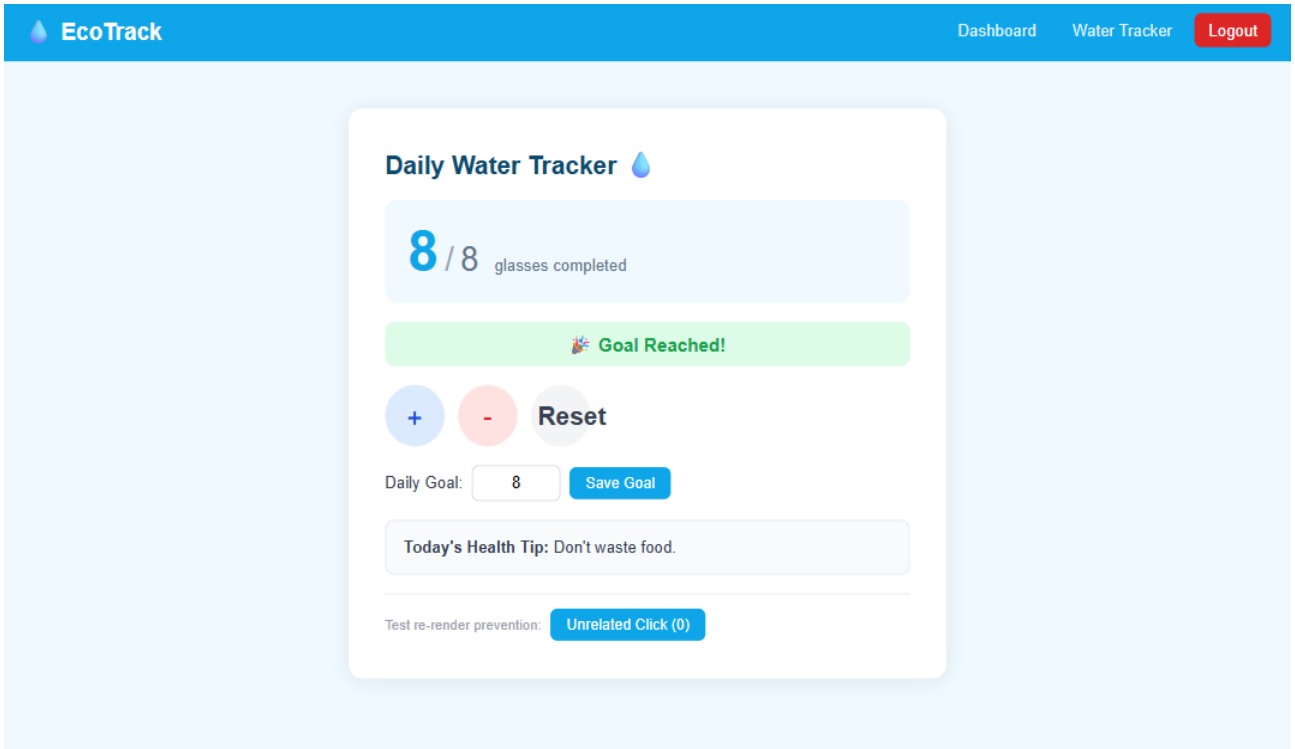




5.2. Water Tracker Page



5.3. Goal Reached state



6. Learning Outcomes:

6.1 React Hooks

- Understood the purpose of useState for managing local component state (count, goal, tip, loading, error).
- Learned useEffect for running side effects — API calls on mount and localStorage sync on state change.
- Applied useCallback to memoize event handler functions and prevent unnecessary child re-renders.

6.2 React Router DOM

- Learned how to define routes using <Routes> and <Route> components.
- Implemented a ProtectedRoute wrapper component that conditionally renders pages based on auth state.
- Used useNavigate hook for programmatic navigation (e.g., redirect after login/logout).

6.3 Performance Optimization

- Understood the concept of unnecessary re-renders and their performance impact.
- Applied React.memo to the CounterDisplay component to prevent it from re-rendering when unrelated state changes.
- Verified optimization by clicking an unrelated button and observing the console — CounterDisplay did not re-render.

6.4 API Integration

- Learned to use the Fetch API to make HTTP GET requests to a third-party REST endpoint.
- Handled asynchronous responses using .then() chains and .catch() for error handling.
- Managed loading and error states to provide feedback to the user during API calls.
- Understood the CORS restriction in browsers and how to use a proxy to work around it.

5.5 Component Architecture

- Learned to split a React application into reusable, single-responsibility components (Navbar, ProtectedRoute, CounterDisplay, HealthTip).
- Separated pages from components and utility logic into distinct folders for better maintainability.
- Understood the concept of prop drilling and how memoization interacts with it.